

Регулярные выражения

- Введение
- Функции регулярных выражений
- Модификаторы регулярных выражений

Введение

Регулярные выражения — это шаблоны с подстановочными знаками, которые применяются для поиска информации в больших наборах данных.

Просто о назначении регулярных выражений

Регулярные выражения (сленг: регулярка) позволяют найти в тексте последовательности, соответствующие шаблону. Разработчик может написать нужный ему шаблон и таким образом **искать в тексте** необходимые символы, набор символов, слова, фрагменты предложений и т.д.

Кроме того, регулярные выражения используют, например, при проверке правильности заполнения следующих текстовых полей:

- домен сайта;
- e-mail;
- банковская карта;
- пароль;
- и пр.

Правильно о назначении регулярных выражений

Регулярное выражение (сокращенное как **regex** или **regexr**; также называется рациональным **выражением**) — это последовательность

символов, определяющая шаблон поиска. Обычно шаблоны используются алгоритмами поиска для выполнения следующих операций:

- найти;
- найти и заменить;
- проверка входных данных.

Самые простые регулярные выражения

Перед тем, как разбираться с регулярными выражениями, приведу небольшой пример использования, на примере поиска соответствия шаблона слову из текста. Допустим, у нас есть входная строка:

"Excludit sanos, Helicone poetas*".

И перед нами стоит задача: проверить наличие слова "sanos" во входной строке.

Для этого, в качестве шаблона регулярного выражения, мы должны просто написать слово "sanos".

example_1. Простой шаблон для поиска

```
<?php

// текст, в котором будем искать совпадение с шаблоном
$cite = "Excludit sanos, Helicone poetas*";

// шаблон для поиска
$pattern = "/sanos/";

// если соответствие шаблону в тексте найдено
if (preg_match($pattern, $cite)) {
    echo "<h3>Вхождение шаблона найдено</h3>";
} else {
```

```
echo "<h3>Вхождение шаблона НЕ найдено</h3>";  
  
}  
  
?>
```

В дальнейшем, при написании шаблонов регулярных выражений, можно указывать не просто какие-то конкретные слова, а составлять сложные комбинации буквенно-цифровых символов.

Синтаксис регулярных выражений

Регулярное выражение представляет собой строку, которая всегда начинается с **символа разделителя**, за ним следует **шаблон** регулярного выражения, затем еще один **символ разделителя** и, наконец, необязательный список **модификаторов**.

```
$regex = "/pechora-pro/i";
```

В приведенном выше примере:

- **/ ... /** — открывающий и закрывающий разделители;
- **pechora-pro** — шаблон, по которому ведется поиск;
- **i** — модификатор, позволяющий вести поиск без учета регистра.

В качестве символа разделителя может быть любой символ, кроме буквы, цифры, обратной косой черты (\) или пробела. Самый распространенный разделитель — это слэш **/**, но, если ваш шаблон сам содержит слэш, удобно выбрать другие разделители, такие как **#** или **~**.

```
$regex = "#pechora-pro#i";
```

```
$regex = "~pechora-pro~i";
```

Функции регулярных выражений

PHP предоставляет программистам множество полезных функций, позволяющих использовать возможности регулярных выражений. Рассмотрим наиболее применяемые из них.

Функция	Описание
preg_match()	Выполняет поиск конкретного образца в некоторой строке. Возвращает 1 (true), если шаблон существует, и 0 (false) в противном случае.
preg_match_all()	Выполняет поиск всех вхождений шаблона в строке. Возвращает количество найденных совпадений с шаблоном в строке, или 0 — если вхождений нет. Функция удобна для поиска и замены.
preg_replace()	Выполняет поиск и замену по регулярному выражению. Работает как со строками, так и с массивами.
preg_split()	Разбивает строку по регулярному выражению. Возвращает массив, состоящий из подстрок заданной входной строки.
preg_grep()	Возвращает массив, состоящий из элементов входящего массива, которые соответствуют заданному шаблону.
preg_last_error	Возвращает код ошибки выполнения последнего регулярного выражения.
preg_last_error_msg	Возвращает сообщение об ошибке последней запущенной функции.

* - полный список функций можно посмотреть в открытых источниках

Рассмотрим основные возможности функций регулярных выражений.

preg_match

preg_match () – выполняет проверку на соответствие регулярному выражению.

Описание

```
preg_match ($pattern, $subject, $matches, $flags, $offset);
```

Ищет в заданном тексте **subject** совпадения с шаблоном **pattern**.

Параметры

- **pattern** – искомый шаблон в виде строки.
- **subject** – входная строка.
- **matches** – необязательный, если указан, будет заполнен результатами поиска. Элемент **\$matches[0]** будет содержать часть строки, соответствующую вхождению всего шаблона.
- **flags** – необязательный. Один, или несколько флагов, меняющих вид массива **matches**, если он присутствует.
- **offset** – необязательный. Обычно поиск осуществляется слева направо, с начала строки. Можно использовать дополнительный параметр **offset** для указания альтернативной начальной позиции для поиска.

Возвращаемые значения

preg_match() возвращает 1, если параметр **pattern** соответствует переданному параметру **subject**, 0 если нет или **false** в случае возникновения ошибки.

Проверяем теорию практикой.

example_2. Вывод массива результатов поиска

```
<?php

// текст, в котором будем искать совпадение с шаблоном

$text = "

    We all live in a yellow submarine

    Yellow submarine, yellow submarine

    We all live in a yellow submarine

    Yellow submarine, yellow submarine
```

```

";

// The Beatles. "Yellow Submarine"

// шаблон для поиска

$pattern = "/submarine/";

// если соответствие шаблону в тексте найдено

if (preg_match($pattern, $text, $matches)) {

    echo "<h3>Вхождение шаблона найдено</h3>";

    // в массив matches будут попадать найденные совпадения

    echo "<pre>";

    print_r($matches);

} else {

    echo "<h3>Вхождение шаблона НЕ найдено</h3>";

}

?>

```

Что означает шаблон для поиска:

```
$pattern = "/submarine/";
```

- набор символов - **submarine**

Именно такой набор символов и именно в такой последовательности (на умении читать и писать шаблоны строится понимание работы с регулярными выражениями).

Посмотрев код листинга, видим, что искомое слово "submarine" найдено один раз. Как так? Там шесть вхождений. Дело в том, что функция preg_match **прекращает** поиск после нахождения **первого вхождения**. Если нужны все вхождения – необходимо использовать функцию preg_match_all.

preg_match_all

preg_match_all ()— выполняет глобальный поиск шаблона в строке.

Описание

```
preg_match_all($pattern, $subject, $matches, $flags, $offset);
```

Ищет в строке **subject** все совпадения с шаблоном **pattern** и помещает результат в массив **matches** в порядке, определяемом комбинацией флагов **flags**.

После нахождения первого соответствия последующие поиски будут осуществляться не с начала строки, а от конца последнего найденного вхождения.

Параметры

- **pattern** – искомый шаблон в виде строки.
- **subject** – входная строка.
- **matches** – необязательный. Массив совпавших значений, отсортированный в соответствии с параметром **flags**.
- **flags** – необязательный. Один, или несколько флагов, меняющих вид массива **matches**, если он присутствует.
- **offset** – необязательный. Обычно поиск осуществляется слева направо, с начала строки. Можно использовать дополнительный параметр **offset** для указания альтернативной начальной позиции для поиска.

Возвращаемые значения

Возвращает количество найденных вхождений шаблона (которое может быть и нулём) или **false** в случае возникновения ошибки.

example_3. Глобальный поиск

```
<?php
```

```
// текст, в котором будем искать совпадение с шаблоном

$text = "

    We all live in a yellow submarine
    Yellow submarine, yellow submarine

    We all live in a yellow submarine
    Yellow submarine, yellow submarine

";

// The Beatles. "Yellow Submarine"

// шаблон для поиска

$pattern = "/yellow submarine/";

// если соответствие шаблону в тексте найдено

if (preg_match_all($pattern, $text, $matches)) {

    echo "<h3>Вхождение шаблона найдено</h3>";

    // в массив matches будут попадать найденные совпадения

    echo "<pre>";

    print_r($matches);

} else {

    echo "<h3>Вхождение шаблона НЕ найдено</h3>";

}

?>
```

Что означает шаблон для поиска:

```
$pattern = "/yellow submarine/";
```

- набор символов - **yellow submarine**

Именно такой набор символов и именно в такой последовательности. Но здесь тоже не все совпадения. Их четыре, куда делись еще два? Все просто, "Yellow submarine" не равно "yellow submarine". Регистр букв имеет значение.

А вот как работать с регистром и не только – читаем дальше.

Модификаторы регулярных выражений

Модификаторы указываются либо в скобках, например так: (?Ui), либо после закрывающего разделителя `/pattern/Ui`.

Модификатор	Описание
i	Выполняет поиск без учета регистра. Например <code>/a/i</code> ищет и <code>a</code> , и <code>A</code> .
m	Выполняет многострочный поиск (шаблоны, которые ищут начало или конец строки, будут соответствовать началу или концу каждой строки)
u	Обеспечивает правильное сопоставление шаблонов в кодировке utf-8 (для поиска русского текста, например).
U	Инвертирует "жадность" (по умолчанию жадный, т.е. пытается захватить как можно большую строку, подходящую по условию).
s	Если используется, то символ точка (<code>.</code>) соответствует и переводу строки (иначе не соответствует).
x	Игнорировать пробелы. В этом случае пробелы нужно экранировать обратным слэшем <code>\</code> .

* - мы рассмотрим только некоторые возможности модификаторов

При использовании модификаторов, можно использовать знак минус (`-`) для отключения модификатора. Например: `(?m-i)` — включаем многострочный поиск и отключаем регистронезависимый.

В следующем примере протестируем модификаторы:

- **i** – поиск без учета регистра;
- **u** – работа с русским текстом.

Обязательно проанализируйте работу всех трех вариантов паттернов (шаблонов).

example_4. Применение модификаторов

```
<?php

// текст, в котором будем искать совпадение с шаблоном

$text = "

    Лес дивен: мрак и глубина.

    Но обещаниям верна

    Душа. И много миль до сна,

    и много миль еще до сна.

";

// Роберт Фрост. "Stopping by Woods on a Snowy Evening"
// перевод Т. Гутиной
// 1. шаблон для поиска

$pattern = "/и много миль/";

// чтобы выбрать все фразы -и много миль
// 2. выполняем поиск без учета регистра
// $pattern = "/и много миль/i";

// 3. включаем кодировку UTF-8 для работы с русским текстом
// $pattern = "/и много миль/iu";

// если соответствие шаблону в тексте найдено

if (preg_match_all($pattern, $text, $matches)) {

    echo "<h3>Вхождение шаблона найдено</h3>";

    // в массив matches будут попадать найденные совпадения

    echo "<pre>";

    print_r($matches);

} else {

    echo "<h3>Вхождение шаблона НЕ найдено</h3>";

}
```

```
?>
```

Что означает шаблон для поиска:

```
$pattern = "/и много миль/i";
```

- набор символов – **и много миль**

Именно такой набор символов, именно в такой последовательности при этом написанных в любом регистре.

preg_split

preg_split () — разбивает строку по регулярному выражению.

Описание

```
preg_split($pattern, $subject, $limit, $flags);
```

Разбивает **строку** в **массив** по регулярному выражению.

Параметры

- **pattern** – строка, содержащая искомый шаблон.
- **subject** – входная строка.
- **limit** – необязательный. Если указан, функция возвращает не более, чем **limit** подстрок. Оставшаяся часть строки будет возвращена в последней подстроке. Специальное значение **limit**, равное -1 или 0, подразумевает отсутствие ограничения.
- **flags** – необязательный. Может быть любой комбинацией дополнительных флагов, влияющих на результат работы функции.

Возвращаемое значение

Возвращает **массив**, состоящий из **подстрок** заданной входной строки **subject**, которая разбита по границам, соответствующим шаблону **pattern** или **false** в случае возникновения ошибки.

example_5. Получение массива из входной строки

```
<?php

    // текст, в котором будем искать совпадение с шаблоном

    $text = <<<HERE

        Когда ж, и где, в какой пустыне,

        Безумец, их забудешь ты?

        Ах, ножки, ножки! где вы ныне?

        Где мнете вешние цветы?

    HERE;

    // Александр Сергеевич Пушкин. "Евгений Онегин"

    // шаблон для поиска (разделения строки)

    // граница разделения - символ переноса строки (\n)

    $pattern = "/\n/";

    // разбиваем строку в массив по шаблону

    $array = preg_split($pattern, $text);

    // массив строк

    echo "<pre>";

    print_r($array);

?>
```

Что означает шаблон для поиска:

```
$pattern = "/\n/";
```

- символ – **\n** (перенос строки)

preg_grep

preg_grep () — возвращает массив вхождений, которые соответствуют шаблону.

Описание

```
preg_grep($pattern, $array, $flags);
```

Возвращает **массив**, состоящий из элементов входящего **массива array**, которые соответствуют заданному шаблону **pattern**.

Параметры

- **pattern** – искомый шаблон в виде строки.
- **array** – входящий массив.
- **flags** – необязательный. В случае, если установлен в PREG_GREP_INVERT, функция возвращает те элементы массива, которые **не соответствуют** заданному шаблону **pattern**.

Возвращаемые значения

Возвращает массив, индексированный ключами из массива **array** или **false** в случае возникновения ошибки.

В следующем примере выберем из входного массива все курсы, где занимаются изучением языка программирования - PHP.

example_6. Отбор элементов входного массива

```
<?php

// массив в котором будем искать совпадение с шаблоном

$array = array(

    "PHP7 для начинающих",

    "Программирование на Visual Basic",

    "Энциклопедия садоводства",

    "Справочник команд php и MySQL",

    "Python для сложных задач",

    "Web программирование. PHP"

);
```

```
// шаблон для поиска

$pattern = "/php/i";

// выбираем из массива по шаблону

// на вход подаем массив

$results = preg_grep($pattern, $array);

// на выходе тоже массив

echo "<pre>";

print_r($results);

?>
```

Что означает шаблон для поиска:

```
$pattern = "/php/i";
```

- набор символов – **php**

Именно такой набор символов, именно в такой последовательности при этом написанных в любом регистре.

preg_replace

preg_replace () — выполняет **поиск** и **замену** по регулярному выражению.

Описание

```
preg_replace($pattern, $replacement, $subject, $limit, $count);
```

Выполняет поиск совпадений в строке **subject** с шаблоном **pattern** и заменяет их на **replacement**.

Список параметров

- **pattern** – искомый шаблон. Может быть как **строкой**, так и **массивом** строк.

- **replacement** – строка или массив строк для замены.
- **subject** – строка или массив строк для поиска и замены.
- **limit** – необязательный. Максимально возможное количество замен каждого шаблона для каждой строки **subject**. По умолчанию - без ограничений.
- **count** – необязательный. Если указан, то эта переменная будет заполнена количеством произведённых замен.

Возвращаемые значения

preg_replace() возвращает **массив**, если параметр **subject** является **массивом**, иначе возвращается **строка**.

Краткая форма записи. Если не учитывать необязательные аргументы, описание функции можно упростить:

```
preg_replace($pattern, $replacement, $subject);
```

Найти и заменить - очень популярная операция с использованием регулярных выражений.

В примерах разберем следующие варианты аргументов функции:

1. **subject** (где ищем) – строка;
 - **pattern** (что ищем) – строка;
 - **replacement** (на что меняем) – строка;
2. **subject** – строка;
 - **pattern** - массив
 - **replacement** – массив;
3. **subject** – массив;
 - **pattern** - массив
 - **replacement** – массив;

Примеры будут ниже.

Начнем с самого простого действия, выполним замену переноса строк, никак не отображаемых браузером, на тег `<br />`.

example_7. Простая операция поиска и замены

```
<?php

    // текст, в котором будем искать совпадение с шаблоном
    $text = "

        Цинизм предполагает общее наличие идеалов.

        Преступление - общее наличие законов.

        Богохульство - общее наличие веры. И так далее.

        А что предполагает убожество?

        Ничего.

    ";

    // Сергей Довлатов. "Соло на ундервуде"

    // вывод исходного текста
    echo "<h2>Исходный текст</h2>";

    echo $text, "<p><hr>";

    // шаблон для поиска определим в самой функции

    // заменим перенос строки \n на тег <br />

    $text = preg_replace("/\n/", "<br />", $text);

    // вывод отформатированного текста

    echo "<h2>Форматированный текст</h2>";

    echo $text;

?>
```

Что означает шаблон для поиска:

```
$pattern = "/\n/";
```

- символ – `\n` (перенос строки)

Значения аргументов функции **preg_replace**:

- **subject** (где ищем): **строка** в переменной \$text;
- **pattern** (что ищем): **строка** "\n";
- **replacement** (на что меняем): **строка** "
".

Строка для замены не обязательно должна содержать простой текст или HTML-код, это могут ссылки на внешние ресурсы, например, изображения.

example_8/index.php. Вставляем в текст изображения

```
<?php

// текст в котором будем искать совпадение с шаблоном

$list = <<<HERE

    {bull} PHP7 для начинающих <br/>

    {bull} Программирование на Visual Basic <br/>

    {bull} Энциклопедия садоводства <br/>

    {bull} Справочник команд php и mySQL <br/>

    {bull} Python для сложных задач <br/>

    {bull} Web программирование. PHP <br/>

HERE;

// шаблон для поиска

$pattern = "{bull}"/";

// строка для замены

$replace = "<img src='bull.png' width='25px'>";

// выполняем поиск и замену

$list = preg_replace($pattern, $replace, $list);

echo $list;

?>
```

Что означает шаблон для поиска:

```
$pattern =("/{bull}"/);
```

- набор символов – **{bull}**

Значения аргументов функции **preg_replace**:

- **subject** (где ищем): **строка** в переменной `$list`;
- **pattern** (что ищем): **строка** "{bull}";
- **replacement** (на что меняем): **строка** "".

preg_last_error

preg_last_error () — возвращает код ошибки выполнения последнего регулярного выражения.

Описание

```
preg_last_error ();
```

Возвращает код ошибки выполнения последнего регулярного выражения.

Параметры

У этой функции нет параметров.

Возвращаемые значения

Возвращает одну из следующих констант:

- PREG_NO_ERROR
- PREG_INTERNAL_ERROR
- PREG_BACKTRACK_LIMIT_ERROR
- PREG_RECURSION_LIMIT_ERROR
- PREG_BAD_UTF8_ERROR
- PREG_BAD_UTF8_OFFSET_ERROR
- PREG_JIT_STACKLIMIT_ERROR

preg_last_error_msg

preg_last_error_msg () — возвращает сообщение об ошибке последней запущенной функции.

Описание

```
preg_last_error_msg();
```

Возвращает сообщение об ошибке последней запущенной функции.

Параметры

У этой функции нет параметров.

Возвращаемые значения

В случае успешного выполнения возвращает сообщение об ошибке. Если ошибки не было, возвращает строку "No error".

В следующем коде задействуем две функции регулярных выражений. Если в предыдущем примере все вхождения шаблона заменялись одним изображением, то теперь напишем сценарий замены каждого вхождения шаблона своим собственным изображением.

Алгоритм решения достаточно простой:

- делаем выборку всех вхождений шаблона во входной строке;
- прогоняем выбранные элементы в цикле. Из простого массива результатов поиска, путем добавления разделителей, делаем массив шаблонов;
- подаем массив на вход функции preg_replace.

Поговорим про темперамент ;)

example_9/index.php. Такая замена будет работать не корректно

```
<?php
    // ЭТО СПИСОК КОТИКОВ...
```

```

// и не вздумайте мне перечить :-[] (я холерик)

$cats = <<<HERE

    Флегматик {pict} <br/>

    Сангвиник {pict} <br/>

    Холерик {pict} <br/>

    Меланхолик {pict} <br/>

HERE;

// шаблон для поиска

$pattern = "{pict}";

// найдем все вхождения шаблона

preg_match_all($pattern, $cats, $matches);

// из массива результатов поиска делаем массив шаблонов

foreach ($matches[0] as $value) {

    $patterns[] = "/" . $value . "/";

}

// echo "<pre>";

// print_r($patterns);

// замена представляет массив тегов изображений

$replace = ["<img src='phlegmatic.jpg'>", "<img
src='sanguine.jpg'>", "<img src='choleric.jpg'>", "<img
src='melancholic.jpg'>"];

// выполняем поиск и замену

$out = preg_replace($patterns, $replace, $cats);

echo $out;

?>

```

Что означает шаблон для поиска:

```
$pattern = "{pict}";
```

- набор символов – **{pict}**

Что означает шаблон поиска и замены:

```
$patterns[] = "/" . $value . "/";
```

- набор символов, динамически формируемых каждой итерацией цикла – **{pict}**, **{pict}**, ...

Значения аргументов функции **preg_replace**:

- **subject** (где ищем): **строка** в переменной \$cats;
- **pattern** (что ищем): **массив** в переменной \$patterns;
- **replacement** (на что меняем): **массив** в переменной \$replace.

Не работает. Точнее сказать, работает не так, как нам бы хотелось. И вот почему.

Два массива на входе preg_replace, в таком случае поиск и замена выполняются следующим образом:

- берем элемент-шаблон из \$patterns[0], находим вхождение во входной строке, заменяем на \$replace[0];
- так для всех элементов массива \$patterns.

Так как массив \$patterns содержит одинаковые элементы ("/{pict}/"), то на первой итерации мы переписывали **все вхождения** шаблона в строку **первым изображением** из массива \$replace. Дальнейшие итерации по массиву бессмысленны (строк для замены "{pict}" во входной строке просто не будет).

Таким образом, для выполнения задачи входная строка должна содержать **уникальные** вхождения шаблонов для замены.

К сведению. Сейчас важнее оптимальности понять принцип работы регулярных выражений и плавно подойти к сложным

примерам.

В следующем примере фрагменты входной строки, подлежащие замене, стали уникальными за счет порядковой нумерации. Нумерация начинается с нуля, совсем не обязательное условие, но в нашем коде так будет легче организовать цикл foreach.

Добавим функцию проверки ошибок.

Примечание. Будьте внимательны при анализе кода. Постарайтесь разобраться с каждой инструкцией предложенного сценария.

example_10/index.php. Тестирование preg_replace с проверкой ошибок

```
<?php

// текст с темпераментами

$temperament = <<<HERE

    Флегматик {pict0} <br/>

    Сангвиник {pict1} <br/>

    Холерик {pict2} <br/>

    Меланхолик {pict3} <br/>

HERE;

// шаблон для поиска

$pattern = "/pict/";

// найдем все вхождения шаблона

preg_match_all($pattern, $temperament, $matches);

// если ошибок поиска вхождений нет, выполняем дальше

if (preg_last_error() == PREG_NO_ERROR) {

    // из массива вхождений сделаем массив шаблонов
```

```

    foreach ($matches[0] as $key => $value) {
        $patterns[] =("/{" . $value . $key . "}/";
    }

    // echo "<pre>";

    // print_r($patterns);

    // замена представляет массив тегов изображений

    $replace = ["<img src='phlegmatic.jpg'>", "<img
src='sanguine.jpg'>", "<img src='choleric.jpg'>", "<img
src='melancholic.jpg'>"];

    // выполняем поиск и замену

    $out = preg_replace($patterns, $replace, $temperament);

    echo $out;

}

?>

```

Что означает шаблон для поиска:

```
$pattern = "/pict/";
```

- набор символов – **pict**

Что означает шаблон поиска и замены:

```
$patterns[] =("/{" . $value . $key . "}/";
```

- набор символов, динамически формируемых каждой итерацией цикла, например – **{pict0}, {pict1}, ...**

Значения аргументов функции **preg_replace**:

- **subject** (где ищем): **строка** в переменной \$temperament;
- **pattern** (что ищем): **массив** в переменной \$patterns;
- **replacement** (на что меняем): **массив** в переменной \$replace.

И опять модернизируем код, **входным аргументом** (subject) теперь будет не строка, а **массив значений**.

example_11/index.php. Все три аргумента функции preg_replace массивы

```
<?php

// массив с темпераментами

$temperament = [

    "Флегматик {pict0}",

    "Сангвиник {pict1}",

    "Холерик {pict2}",

    "Меланхолик {pict3}"

];

// шаблон для поиска

$pattern = "/pict/";

// найдем все вхождения шаблона

// можно просто подсчитать количество элементов массива
count($temperament)

// или так...

preg_match_all($pattern, implode($temperament), $matches);

// если ошибок поиска вхождений нет, выполняем дальше

if (preg_last_error() == PREG_NO_ERROR) {

    // из массива вхождений сделаем массив шаблонов

    foreach ($matches[0] as $key => $value) {

        $patterns[] = "{ " . $value . $key . " }";

    }

    // echo "<pre>";

    // print_r($patterns);

    // замена представляет массив тегов изображений
```



```

$replace = ["<img src='phlegmatic.jpg'>", "<img
src='sanguine.jpg'>", "<img src='choleric.jpg'>", "<img
src='melancholic.jpg'>"];

// выполняем поиск и замену

// все три параметра - массивы

$results = preg_replace($patterns, $replace,
$temperament);

// если входные данные - массив, то функция тоже

// возвращает массив

echo "<pre>";

print_r($results);

}

?>

```

Что означает шаблон для поиска:

```
$pattern = "/pict/";
```

- набор символов – **pict**

Что означает шаблон поиска и замены:

```
$patterns[] =("/{" . $value . $key . "}/");
```

- набор символов, динамически формируемых каждой итерацией цикла, например – **{pict0}, {pict1}, ...**

Значения аргументов функции **preg_replace**:

- **subject** (где ищем): **массив** в переменной \$temperament;
- **pattern** (что ищем): **массив** в переменной \$patterns;
- **replacement** (на что меняем): **массив** в переменной \$replace.